

johnthfoster/bspline

Complete Repository Anatomy & Usage Guide

Every function explained · Mapped to Assignments 4, 5 & 6

Based on the actual installed source of the bspline package (bspline.py + splinelab.py). Every class, method, and function is documented with what it does, what it returns, and exactly where in your physics code you should call it.

Contents

1. Repository Layout — What Files Exist and Why
2. The Two Modules at a Glance
3. Module bspline.py — Class Bspline, Method by Method
 - 3.1 `__init__` — Creating a spline basis
 - 3.2 `__call__` — Evaluating all basis functions at a point
 - 3.3 `d()` — First derivative of all basis functions
 - 3.4 `diff()` — Any-order derivative as a callable
 - 3.5 `collmat()` — The collocation matrix (KEY for Assignment 4)
 - 3.6 `plot()` / `dplot()` — Quick visualisation
4. Module splinelab.py — Utility Functions
 - 4.1 `augknt()` — Build the knot vector (you call this first)
 - 4.2 `aptknt()` — Alternative knot vector from collocation sites
 - 4.3 `aveknt()` — Running average of knot entries
 - 4.4 `knt2mlt()` — Count knot multiplicities
 - 4.5 `spcol()` — Full collocation matrix with derivative rows
5. The memoize Decorator — Why the Library is Fast
6. IMPORTANT: The Order Convention — p vs k
7. Putting It All Together — Assignment 4 (Poisson Collocation)
8. Putting It All Together — Assignment 5 (Galerkin Eigenvalue)
9. Putting It All Together — Assignment 6 (SCF Loop)
10. Installation and Import Patterns
11. What the Library Does NOT Do (and what you write yourself)

1. Repository Layout — What Files Exist and Why

johntrfoster/bspline/

```
■
■■■ bspline/                ← the actual Python package (this is what pip installs)
■   ■■■ __init__.py         ← makes 'bspline' importable; imports Bspline class
■   ■■■ bspline.py          ← THE MAIN FILE: class Bspline + memoize decorator
■   ■■■ splinelab.py        ← UTILITY FILE: augknt, aptknt, aveknt, knt2mlt, spcol
■
■■■ test/                   ← unit tests (useful to read to understand expected behaviour)
■
■■■ README.md               ← usage examples (what you read on GitHub)
■■■ setup.py                ← pip packaging metadata
■■■ CHANGELOG.md            ← version history
■■■ LICENSE.md              ← MIT licence (free to use in coursework)
```

When you run **pip install bspline**, Python copies the **bspline/** sub-folder into your environment. After that you have two importable modules:

```
import bspline                # gives you:  bspline.Bspline
import bspline.splinelab     # gives you:  bspline.splinelab.augknt  etc.

# Common shorthand used in the docs:
import bspline.splinelab as splinelab
```

2. The Two Modules at a Glance

Module	What it contains	You use it for
bspline.py	class Bspline (the core object)	Evaluate basis functions and derivatives at any point x ; build collocation matrices
splinelab.py	5 utility functions	Build knot vectors (augknt); generate collocation matrices with multiple-derivative rows (sp

3. Module bspline.py — Class Bspline, Method by Method

3.1 `__init__(knot_vector, order)` — Creating a spline basis

What it does: Stores the knot vector and spline order. Runs two dummy evaluations (`__call__` and `d` at $x=0$) to pre-populate the memoisation cache for speed.

Parameters:

- **knot_vector** — the full augmented knot vector (with k repeated endpoints). This is the array you get back from `splinelab.augknt()`.
- **order** — called **p** internally. This is the polynomial degree, NOT the order in the assignment's sense. For cubic B-splines: **order = 3** (degree 3 = polynomial of degree 3). The assignment PDF uses $k=4$ for the same thing. See Section 6 for the full explanation.

```
import numpy as np
import bspline
import bspline.splinelab as splinelab
```

[illegible]

```
r_max = 20.0
n_pts = 12          # number of distinct physical knot points
```

```
inner = np.linspace(0, r_max, n_pts) # [0, 1.8, 3.6, ..., 20.0]
k_vec = splinelab.auqknt(inner, p)   # add p repeated endpoints
```

```
B = bspline.Bspline(k_vec, p)          # create the basis object
```

```
# After this, B is ready to evaluate at any x in [0, 20].
# Number of basis functions = len(k_vec) - p - 1
n_basis = len(k_vec) - p - 1
print(f"Number of B-spline basis functions: {n_basis}")
# = n_pts + p - 2      (for a clean linear sequence)
```

■ The library uses $\text{order}=p$ (degree). The assignment PDF uses $k = p+1 = \text{order}+1$. Always check which convention you are using before building your matrix. See Section 6 for a full comparison table.

3.2 B(x) — Evaluate ALL basis functions at one point x

What it returns: A 1-D NumPy array of length `n_basis`. The i -th element is $B_i(x)$.

When to use it: When you want to know which B-splines are non-zero at a specific point x . This is the inner loop workhorse for building matrices row by row.

```
vals = B(0.5)          # evaluate ALL basis functions at x = 0.5
# vals is a 1-D array, length = n_basis
# vals[i] = value of the i-th B-spline at x=0.5
```

```
print(vals)
# Most entries will be 0.0 because each B-spline is locally supported.
# At most p+1 = 4 entries are non-zero at any point (for cubic splines).
```

```
# To reconstruct a function f(x) from coefficients c:
```

```
# Or using the collocation matrix (see 3.5):
A = B.collmat([0.5])
f_at_x = np.sum(A * c, axis=-1) # same result
```

3.3 B.d(x) — First derivative of ALL basis functions at x

When to use it: This is the *fast path* for first derivatives. Under the hood it calls `__basis` with `compute_derivatives=True`, which replaces the numerator in the last recursion step with $\pm p$ — the standard Cox-de Boor differentiation formula.

[illegible]

3.4 B.diff(order=n) — Returns a callable for the n-th derivative

[illegible]

■ `B.diff(order=1)(x)` gives the SAME result as `B.d(x)` but is slower because it does not use the fast internal path. Always use `B.d(x)` when you only need first derivatives.

What it returns: A 2-D NumPy array of shape $(\text{len}(\text{tau}), n_basis)$.

In plain words: row i contains the values (or derivatives) of ALL basis functions evaluated at the collocation site $\tau[i]$.

```
import numpy as np
import bspline
import bspline.splinelab as splinelab
```

[illegible]

```
# Matrix of function values (for reconstructing V later):
A0 = B.collmat(tau, deriv_order=0)
# A0[i, j] = B_j(tau[i])
```

```
# Row 0: boundary condition  $\phi(r=0) = 0$ 
# Only B_0 is nonzero at  $r=0$  with augmented knots, so  $A[0,0]=1$ 
A_full[0, 0] = 1.0
```

```
# Build RHS
rhs = np.zeros(n_basis)
rhs[0] = 0.0 #  $\phi(0) = 0$ 
```

```

for i, ri in enumerate(tau):
    rhs[i+1] = -ri * 4*np.pi * rho_function(ri) # Poisson
rhs[-1] = Q_total                               # phi(inf) = Q

# Solve
from scipy.linalg import lu_factor, lu_solve
lu, piv = lu_factor(A_full)
c       = lu_solve((lu, piv), rhs)

# Reconstruct phi(r) = sum_j c_j B_j(r)
r_plot = np.linspace(0.05, 15, 300)
A_plot = B.collmat(r_plot, deriv_order=0) # shape (300, n_basis)
phi     = A_plot @ c                      # shape (300,) matrix-vector product
V       = phi / r_plot                    # V(r) = phi(r)/r

```

■ **KEY:** `collmat(tau)` builds the full matrix in one vectorised call. This is much faster than a Python loop over collocation points. The shape is always `(n_collocation_points, n_basis_functions)`.

3.6 `B.plot()` / `B.dplot()` — Quick visualisation

What they do: Plot all basis functions (or their first derivatives) over the full knot range on a fine grid of 1000 points, using matplotlib. These are convenience methods — you would normally use your own plot code for publication-quality figures.

```

# Quick sanity check: do your B-splines look right?
B.plot()      # shows all B_i(x) – should see overlapping bumps summing to 1
B.dplot()     # shows all dB_i/dx – derivatives of the above

# For your own plot (more control):
import matplotlib.pyplot as plt
x = np.linspace(0, r_max, 500)
A = B.collmat(x) # shape (500, n_basis)
for j in range(A.shape[1]):
    plt.plot(x, A[:, j], alpha=0.6)
plt.title("All B-splines"); plt.show()

```


4. Module splinelab.py — Utility Functions

4.1 augknt(knots, order) — Build the knot vector [CALL THIS FIRST]

What it does: Takes your list of distinct physical knot points and prepends/appends **order** copies of the first/last point. This is mandatory before creating a Bspline object.

Parameters:

- **knots** — 1-D array of distinct, sorted physical positions, e.g. `np.linspace(0, 20, 30)`. The endpoints (0 and 20 here) are already in this array; `augknt` adds *extra* copies.
- **order** — the spline degree p (= assignment k minus 1).

Returns: 1-D array of length $\text{len}(\text{knots}) + 2 * (\text{order})$. Has $(\text{order} + 1)$ copies at each end.

[illegible]

■ *This is the ONLY function from splinelab you MUST call. Everything else is optional.*

4.2 aptknt(tau, order) — Knot vector from collocation sites

What it does: Instead of a uniform grid, you supply the collocation sites directly and `aptknt` works out an appropriate knot vector. This is the MATLAB `aptknt` equivalent.

When to use it: If you have a specific set of points where you want the equation to be satisfied and you want the knot vector to match those sites as closely as possible. For your assignments, **augknt is simpler and**

preferred.

```
import numpy as np
import bspline.splinelab as splinelab

p = 3
tau = np.linspace(0, 20, 15) # 15 collocation sites
t = splinelab.aptknt(tau, p)
B = bspline.Bspline(t, p)

# aptknt internally calls aveknt on the interior sites
# to place knots at running averages of the collocation points.
# This gives better interpolation properties than a uniform grid
# but the difference is small for physics applications.
```

4.3 aveknt(t, k) — Running average of k successive knot entries

What it does: Computes the running average of k successive entries of array t. Used internally by aptknt. You will rarely need to call this directly.

```
import bspline.splinelab as splinelab
import numpy as np

t = np.array([0., 1., 2., 3., 4., 5.])
print(splinelab.aveknt(t, 3))
# Averages of 3 successive entries:
# [(0+1+2)/3, (1+2+3)/3, (2+3+4)/3, (3+4+5)/3]
# = [1.0, 2.0, 3.0, 4.0]
```

4.4 knt2mlt(t) — Count multiplicities of knot values

What it does: Returns an array where each element counts how many times that knot value has appeared previously. Used internally by spcol to know which derivative order to use at each site.

```
import bspline.splinelab as splinelab
import numpy as np

t = np.array([0., 0., 0., 0., 2., 4., 6., 8., 10., 10., 10., 10.])
m = splinelab.knt2mlt(t)
print(m)
# [0, 1, 2, 3, 0, 0, 0, 0, 0, 1, 2, 3]
# The first 0 has multiplicity 0 (first time seen)
# The second 0 has multiplicity 1 (seen once before)
# etc.
```

4.5 spcol(knots, order, tau) — Collocation matrix with multiplicity-aware derivatives

What it does: Like collmat() but handles sites where the same point appears multiple times, using a higher-order derivative for each repeated occurrence. This is the MATLAB spcol equivalent.

When to use it: When you want to enforce both a function value and its derivative at the same point. For standard Poisson collocation (Assignment 4) with distinct collocation points, collmat() is simpler.

```
import numpy as np
import bspline
import bspline.splinelab as splinelab

p = 3
```

```

inner = np.linspace(0, 10, 12)
t      = splinelab.augknt(inner, p)

# Distinct collocation sites (function value only)
tau = np.unique(t)[1:-1]          # interior sites, each appearing once

A = splinelab.spcol(t, p, tau)    # equivalent to B.collmat(tau, deriv_order=0)
                                   # since all multiplicities = 0

# With repeated sites (enforcing value + derivative at same point):
tau_repeated = np.array([2., 2., 5., 5.]) # each site twice
A2 = splinelab.spcol(t, p, tau_repeated)
# Row 0: B_j(2.)          (0th derivative, multiplicity 0)
# Row 1: dB_j/dx(2.)     (1st derivative, multiplicity 1)
# Row 2: B_j(5.)
# Row 3: dB_j/dx(5.)

```

5. The memoize Decorator — Why the Library is Fast

The **memoize** class is a caching decorator applied to `__call__` and `d`. The first time you call `B(0.5)`, the result is computed and stored in a dictionary on the `Bspline` instance. The second call to `B(0.5)` retrieves the cached result in $O(1)$ time — no recomputation.

```
# Inside bspline.py (simplified):
class memoize:
    def __call__(self, *args, **kw):
        obj    = args[0]                # the Bspline instance
        key    = (self.func, args[1:]) # the function + arguments
        cache  = obj.__cache            # per-instance dictionary
        if key not in cache:
            cache[key] = self.func(*args, **kw) # compute once
        return cache[key]                # return cached

# @memoize is applied to __call__ and d:
@memoize
def __call__(self, xi):
    return self.__basis(xi, self.p, compute_derivatives=False)
```

The memoize cache means:

- If you call `B(ri)` multiple times for the same `ri` (e.g. once per matrix row), it is computed only once.
- This is especially helpful in the Galerkin integration loop where the same Gauss points appear across many matrix element calculations.

■ *The cache lives on the `Bspline` instance. If you create a new `Bspline(t, p)` object (e.g. inside a loop), the cache starts empty each time. Create your `Bspline` object **ONCE** and reuse it.*

6. IMPORTANT: The Order Convention — Library p vs Assignment k

This is the single most confusing aspect of using this library with your assignments. The assignment PDF and the library use different names for the same concept.

Property	Assignment PDF (de Boor / physics convention)	johnfoster/bspline (CS convention)
Order symbol	k	p (called 'order' in <code>__init__</code>)
Polynomial degree	k - 1	p (same as the 'order' parameter)
Cubic splines	k = 4	p = 3
Knot repeats at boundary	k copies	(p + 1) copies (augknt adds p, knots has 1)
Number of B-splines (given N total knots incl. ghosts)	N - k	len(t) - p - 1
Minimum for 2nd derivative	k >= 4 (i.e. cubic)	p >= 3 (i.e. cubic)

■ **KEY: Whenever the assignment says k=4, pass p=3 to the library. Whenever you count B-splines from the library, use len(t) - p - 1, not N - k.**

```
# TRANSLATION TABLE:
#
# Assignment says:      Library call:
# k = 4 (order)  ->    p = 3 (degree)
# k = 5          ->    p = 4
#
# Assignment formula:   Library formula:
# N - k (n_splines) -> len(t) - p - 1
# t[i+k-1] - t[i]  ->  t[i+p] - t[i]    (shifted by 1 due to 0-indexing)
#
# Concrete example:
import numpy as np
import bspline, bspline.splinelab as splinelab

# Using ASSIGNMENT notation (k=4):
k_assign = 4
inner = np.linspace(0, 20, 30)
# Manual augmentation:
t_manual = np.concatenate([np.repeat(inner[0], k_assign),
                             inner[1:-1],
                             np.repeat(inner[-1], k_assign)])
n_sp_assignment = len(t_manual) - k_assign # assignment formula

# Using LIBRARY notation (p=3):
p_lib = k_assign - 1 # = 3
t_lib = splinelab.augknt(inner, p_lib)
n_sp_library = len(t_lib) - p_lib - 1 # library formula

print(n_sp_assignment, n_sp_library) # should be equal
```

Here is the complete pattern using the library correctly, with every library call annotated.

```
rhs[-1] = Q # phi(inf) = Q
```

[illegible]

8. Putting It All Together — Assignment 5 (Galerkin Eigenvalue)

```
import numpy as np
import bspline
import bspline.splinelab as splinelab
from scipy.linalg import eigh
from numpy.polynomial.legendre import leggauss

p_lib = 3
Z = 1      # hydrogen
ell = 0
r_max = 30.0
N_pts = 50
ngl = 5    # Gauss-Legendre points per interval

inner = np.linspace(0, r_max, N_pts)
t = splinelab.augknt(inner, p_lib)
B = bspline.Bspline(t, p_lib)      # CREATE ONCE
n_basis = len(t) - p_lib - 1

# Active (interior) basis functions: drop first and last
# (indices 1 to n_basis-2 in 0-based indexing)
n_act = n_basis - 2

# Gauss-Legendre nodes and weights on [-1, 1]
gl_z, gl_w = leggauss(ngl)

H_mat = np.zeros((n_act, n_act))
S_mat = np.zeros((n_act, n_act))

# Distinct knot intervals
t_uniq = np.unique(t)

for m in range(len(t_uniq) - 1):
    a, b = t_uniq[m], t_uniq[m+1]
    if a == b: continue

    # Transform Gauss nodes to [a, b]
    x_pts = 0.5*(b-a)*gl_z + 0.5*(b+a)    # Gauss points in [a, b]
    w_pts = 0.5*(b-a)*gl_w                # scaled weights

    # Evaluate ALL basis functions and their derivatives
    # at ALL Gauss points in this interval using collmat
    # collmat(x_pts) -> shape (ngl, n_basis)
    # collmat(x_pts, deriv=1) -> shape (ngl, n_basis)
    Bvals = B.collmat(x_pts, deriv_order=0) # B_j(x_k)
    dBvals = B.collmat(x_pts, deriv_order=1) # dB_j/dx at x_k

    # Loop over active basis function pairs (ii, jj) -> matrix indices
    for ii in range(n_act):
        i = ii + 1 # actual basis index (1-based active)
        for jj in range(n_act):
            j = jj + 1

            Bj = Bvals[:, j] # shape (ngl,) - B_j at Gauss points
            Bi = Bvals[:, i] # shape (ngl,) - B_i at Gauss points
            dBj = dBvals[:, j]
            dB_i = dBvals[:, i]
```



```

# Kinetic:  $0.5 \cdot \int dB_j \cdot dB_i \, dr$ 
kin = np.sum(w_pts * 0.5 * dBj * dBi)

# Centrifugal:  $\ell(\ell+1)/2 \cdot \int B_j/r^2 \cdot B_i \, dr$ 
cent = np.sum(w_pts * 0.5 * ell * (ell+1) / x_pts**2 * Bj * Bi)

# Coulomb:  $-Z \cdot \int B_j/r \cdot B_i \, dr$ 
coul = np.sum(w_pts * (-Z) / x_pts * Bj * Bi)

# Overlap:  $\int B_j \cdot B_i \, dr$ 
over = np.sum(w_pts * Bj * Bi)

H_mat[ii, jj] += kin + cent + coul
S_mat[ii, jj] += over

# Solve generalised eigenvalue problem:  $Hc = ES c$ 
eigenvalues, eigenvectors = eigh(H_mat, S_mat)

# Print bound states
for idx, E in enumerate(eigenvalues[:8]):
    if E >= 0: break
    n = idx + ell + 1
    print(f"n={n}, ell={ell}: E = {E:.6f} Ha (exact: {-Z**2/(2*n**2):.6f} Ha)")

```

■ The key insight: `B.collmat(x_pts)` returns a matrix of shape (n_{gl}, n_{basis}) . Column j gives B_j evaluated at all Gauss points in one shot. This vectorises the inner loop completely — no Python loop over Gauss points.

Assignment 6 reuses Assignments 4 and 5 almost unchanged. The only additions are: computing rho from wave functions, adding V_{ee} to the Hamiltonian matrix elements, and iterating.

[illegible]

10. Installation and Import Patterns

[illegible]

11. What the Library Does NOT Do (and What You Write Yourself)

The library handles B-spline evaluation and matrix construction. Everything physics-specific is yours to implement.

You need to do this yourself	Library provides
Define the charge density $\rho(r)$	B-spline evaluation at any point
Set boundary condition rows in matrix A	<code>collmat()</code> for second derivatives
Build the RHS vector $-r*4\pi\rho$	<code>augknt()</code> for knot vector
Call <code>scipy</code> to solve $A c = b$	All basis functions in one array call
Build Galerkin H and S matrices (loop over knot intervals)	<code>d(x)</code> for first derivatives
Implement Gauss-Legendre integration loop	<code>diff(order=2)</code> for second derivatives
Normalise eigenvectors	<code>plot()</code> for quick visualisation
Compute ρ from wave functions (Assign 6)	(nothing — physics is yours)
SCF convergence loop (Assign 6)	(nothing — physics is yours)

■ *The library does about 20% of the work — the elegant, fiddly B-spline maths. Your 80% is the physics: boundary conditions, PDEs, integrands, and iterative loops.*

That is every piece of the repository, explained in the context of your assignments. Start with **augknt** → **Bspline** → **collmat** for Assignment 4, add **d()** and **Gauss quadrature** for Assignment 5, and wrap everything in a convergence loop for Assignment 6.